



# MySQL - RDBMS

Trainer: Nisha Dingare



# Codd's rules

---

- Proposed by Edgar F. Codd – pioneer of the RDBMS – in 1970.
- If any DBMS follow these rules, it can be considered as RDBMS.
- The 0<sup>th</sup> rule is the main rule known as “The foundation rule”.
  - For any system that is advertised as, or claimed to be, a relational data base management system, that system must be able to manage data bases entirely through its relational capabilities.
- The rest of rules can be considered as elaboration of this foundation rule.



# Codd's rules

## Rule 1: The Information Rule

### Statement:

All information in a relational database is represented explicitly at the logical level and in exactly one way — by values in tables.

### Explanation:

This rule means that **all data must be stored in table form** — rows and columns — and not hidden in other formats such as files, pointers, or structures outside the database tables.

---

## Rule 2: The Guaranteed Access Rule

### Statement:

Each and every datum (atomic value) in a relational database is guaranteed to be logically accessible by resorting to a combination of table name, primary key value, and column name.

### Explanation:

This rule ensures **direct and unique access** to every single piece of data in the database.

You can locate any data value using:

**Table name** — which table the data is in,

**Primary key** — which row it's in,

**Column name** — which field (attribute) it's in.



# Codd's rules

## Rule 3: Systematic Treatment of Null Values



### Statement:

Null values (distinct from the empty character string or a string of blank characters and distinct from zero or any other number) are supported in a fully relational DBMS for representing missing information and inapplicable information in a systematic way, independent of data type.



### Explanation:

This rule ensures that the database can represent **unknown or inapplicable data** in a consistent and meaningful way, regardless of the data type.

**NULL**  $\neq$  0

**NULL**  $\neq$  empty string ("")

**NULL** simply means *value not known or not applicable*.

The database must be able to handle NULLs consistently in all operations (like comparisons, aggregations, joins, etc.) and across all data types (numeric, text, date, etc.).



# Codd's rules

## Rule 4: Dynamic Online Catalog Based on the Relational Model



### Statement:

The database description is represented at the logical level in the same way as ordinary data, so that authorized users can apply the same relational language to its interrogation as they apply to the regular data.



### Explanation:

This rule states that **the metadata** (data about the database structure — such as table definitions, column types, constraints, etc.) must also be stored **in tables** — just like normal user data.

That means:

Information about the database's structure should be accessible using **the same query language (e.g., SQL)** used to query normal data.

There should not be any special tools or commands required to get this information.

This ensures **consistency** and **transparency** — users can query the system catalog to find out how the database is organized.



# Codd's rules

## Rule 5: The Comprehensive Data Sublanguage Rule

### Statement:

A relational system may support several languages. However, there must be at least one language that supports all functionalities of an RDBMS, i.e., data definition, data manipulation, integrity constraints, transaction management, and authorization.

### Explanation:

This rule says that a true RDBMS must provide **one comprehensive language** (like SQL) that can handle **all database tasks** — not just data querying.

That single language should be able to:

**Define data structures** – Create or modify tables, schemas, etc. (Data Definition Language – DDL)

**Manipulate data** – Insert, update, delete, or query data (Data Manipulation Language –/ DML)

**Specify integrity constraints** – Enforce data validity

**Manage transactions** – Commit or roll back changes safely

**Handle authorization** – Control user permissions



# Codd's rules

## Rule 6: The View Updating Rule



### Statement:

All views that are theoretically updatable are also updatable by the system.



### Explanation:

A **view** in a relational database is a **virtual table** derived from one or more base tables using a query.

This rule says that **if a view can be logically updated**, the system (DBMS) should allow it to be updated directly.

That means — if a change made to a view can be uniquely and unambiguously translated into a change in the base table(s), the DBMS should support that update.

---

## Rule 7: High-Level Insert, Update, and Delete



### Statement:

The capability of handling a base relation or a derived relation as a single operand applies not only to the retrieval of data but also to the insertion, update, and deletion of data.



### Explanation:

This rule states that the **same relational operations** available for retrieving data (like working with whole tables or sets of rows) should also be available for **modifying data** — not just one row at a time.

That means the database should allow **set-level operations** (bulk inserts, updates, deletes) rather than forcing users to work record-by-record.



# Codd's rules

## Rule 8: Physical Data Independence



### Statement:

Application programs and terminal activities remain logically unbroken whenever any changes are made in either storage representations or access methods.



### Explanation:

**Physical data independence** means that **changes in how data is stored physically** (like file formats, indexing methods, storage devices, etc.) should **not affect how users or programs access the data**.

In other words:

You can reorganize the database storage,

Move it to a new hardware system, or

Change the indexing mechanism

without breaking queries, applications, or user interfaces.



# Codd's rules

## Rule 9: Logical Data Independence



### Statement:

Application programs and terminal activities remain logically unbroken when information-preserving changes of any kind that theoretically permit un-impairment are made to the base tables.



### Explanation:

**Logical data independence** means that **changes in the logical structure of the database (schema)** should **not require changes in the application programs** that use the database — **as long as the information remains the same.**

In simpler terms:

If you modify how data is organized logically (for example, splitting or combining tables),

But you do **not lose or alter** the meaning of the data,

Then your applications and user queries should still work as before.

It focuses on **protecting users and applications** from changes at the **logical level**, just as **Rule 8 (Physical Data Independence)** protects them from changes at the physical level.



# Codd's rules

## Rule 10: Integrity Independence



### Statement:

Integrity constraints specific to a particular relational database must be definable in the relational data sublanguage and storable in the catalog, not in the application programs.



### Explanation:

This rule says that **integrity constraints** — the rules that ensure the **accuracy and consistency of data** — should be **defined inside the database itself**, not buried within external application code.

Integrity constraints must:

Be **expressed using the database language** (like SQL).

Be **stored in the system catalog** (metadata tables).

Be **enforced automatically** by the database, not by manual checks in programs.

This allows the database to maintain data integrity consistently across all applications.

Eg:

PRIMARY KEY, NOT NULL, CHECK, and FOREIGN KEY are integrity constraints defined on emp table.

They're stored in the data dictionary/catalog and automatically enforced. Any program inserting invalid data will be rejected by the DBMS — no need for custom checks in the application layer.



# Codd's rules

## Rule 11: Distribution Independence



### Statement:

The end user must not be able to see that the data is distributed over various locations. Users should always get the impression that the data is located at one site only.



### Explanation:

This rule refers to **data distribution transparency**.

It means that even if data is physically stored across multiple locations — different servers, databases, or even geographic regions — the user should interact with it **as if it's all in one place**.

The DBMS should manage:

Data distribution,

Replication,

Data movement,

**without affecting** how users query or manipulate data.



# Codd's rules

## Rule 12: The Non-Subversion Rule

### Statement:

If a relational system has a low-level (single-record-at-a-time) language, that low level cannot be used to subvert or bypass the integrity rules and constraints expressed in the higher-level relational language (multiple-records-at-a-time).

### Explanation :

This rule ensures that **no matter how data is accessed**, all the **integrity rules and constraints** defined at the database level **must always be enforced**.

In other words:

A relational database may allow **different ways to access data** — for example, SQL (high-level) and APIs or procedural code (low-level).

However, **no access method** should be able to **bypass the data integrity rules** (like primary keys, foreign keys, or checks).

The system must **protect itself** from being corrupted by low-level operations.



# Normalization

- Concept of table design: Table, Structure, Data Types, Width, Constraints, Relations.
- Goals:
  - Efficient table structure.
  - Avoid data redundancy i.e. unnecessary duplication of data (to save disk space).
  - Reduce problems of insert, update & delete.
- Done from input perspective.
- Based on user requirements.
- Part of software design phase.
- View entire appln on per transaction basis & then normalize each transaction separately.
- Transaction Examples:
  - Banking, Rail Reservation, Online Shopping.



# Normalization

- For given transaction make list of all the fields.
- Strive for atomicity.
- Get general description of all field properties.
- For all practical purposes we can have a single table with all the columns. Give meaningful names to the table.
- Assign datatypes and widths to all columns on the basis of general desc of fields properties.
- Remove computed columns.
- Assign primary key to the table.
- At this stage data is in un-normalized form.
- UNF is starting point of normalization.
- UNF suffers from
  - Insert anomaly
  - Update anomaly
  - Delete anomaly



# Normalization – 1 NF

---

- 1. Remove repeating group into a new table.
- 2. Key elements will be PK of new table.
- 3. (Optional) Add PK of new table to Original table to give us Composite PK.
  - Repeat steps 1-3 infinitely -- to remove all repeating groups into new tables.
  - This is **1-NF**. No repeating groups present here. One to Many relationship between two tables.



# Normalization – 2 NF

- 4. Only table with composite PK to be examined.
- 5. Those columns that are not dependent on the entire composite PK, they are to be removed into a new table.
- 6. The key elements on which the non-key elements were originally dependent, it is to be added to the new table, and it will be the PK of new table.
  - Repeat steps 4-6 infinitely -- to separate all non-key elements from all tables with composite primary key.
  - This is **2-NF**. Many-to-Many relationship.



# Normalization – 3 NF

- 7. Only non-key elements are examined for inter-dependencies.
- 8. Inter-dependent cols that are not directly related to PK, they are to be removed into a new table.
- 9. (a) Key ele will be PK of new table.
- 9. (b) The PK of new table is to be retained in original table for relationship purposes.
  - Repeat steps 7-9 infinitely to examine all non-key eles from all tables and separate them into new table if not dependent on PK.
  - This is **3-NF**.



# Normalization – 4NF (BCNF)

---

- To ensure data consistency (no wrong data entered by end user).
- Separate table to be created of well-known data. So that min data will be entered by the end user.
- This is BCNF (Boyce-Codd) NF or 4-NF.



# De-normalization

---

- Normalization will yield a structure that is non-redundant.
- Having too many inter-related tables will lead to complex and inefficient queries.
- To ensure better performance of analytical queries, few rules of normalization can be compromised.
- This process is de-normalization.





Thank you!

